ConsumerAuthorization

Interface Design Description

Eclipse Arrowhead 5.2.2 | core system | Revision 2026-11-09 | REVIEW DRAFT

Purpose and document structure

This document consolidates the system interface designs. Detailed Java service/transport IDDs are retained as separate appendices, followed by available Go and Rust interface descriptions. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-core-java-spring/consumer-authorization. 11 detailed Java IDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

Go implementation

Primary folder: Eclipse-Arrowhead-go/ah5-core-go/consumer-authorization.

ConsumerAuthorization - Go 5.2.2

Standalone core system, reorganized on 2026-10-28 following the Eclipse-Arrowhead-java distribution layout. Build and operate from this folder; no original Arrowhead checkout or sibling source module is required. The executable is bin/ConsumerAuthorization. Entry point: ./cmd/consumer-authorization. Module: arrowhead/ah5-core-go/consumer-authorization. See DEPLOYMENT.md for configuration and standalone container use. Existing compatibility limits remain in the copied upstream documentation and UPSTREAM-README.md. Existing role selection and API behavior are retained. Documentation and license notices are included in this folder. Source provenance and import-only rewrites are listed in the distribution's SOURCE-MANIFEST.json. No running cloud or upstream source was changed.

Listening port

Built-in default 127.0.0.1:8445 (Arrowhead default). The shipped config.json sets "listen": "0.0.0.0:8445". Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../../LISTENING-PORTS.md) for precedence and all systems.

Detailed implementation evidence

0 native Go IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Go HTTP route inventory

This is the repository route inventory, not a new differential test. Route presence does not establish DTO, error, authorization or persistence equivalence.

- DELETE /consumerauthorization/authorization-token/encryption-key | Go route recorded: True

- POST /consumerauthorization/authorization-token/encryption-key | Go route recorded: True

- POST /consumerauthorization/authorization-token/generate | Go route recorded: True

- GET /consumerauthorization/authorization-token/public-key | Go route recorded: True

- GET /consumerauthorization/authorization-token/token/verify/{token} | Go route recorded: True

- POST /consumerauthorization/authorization/grant | Go route recorded: True

- POST /consumerauthorization/authorization/lookup | Go route recorded: True

- POST /consumerauthorization/authorization/mgmt/check | Go route recorded: True

- POST /consumerauthorization/authorization/mgmt/grant | Go route recorded: True

- POST /consumerauthorization/authorization/mgmt/query | Go route recorded: True

- DELETE /consumerauthorization/authorization/mgmt/revoke | Go route recorded: True

- DELETE /consumerauthorization/authorization/mgmt/token/encryption-key | Go route recorded: True

- POST /consumerauthorization/authorization/mgmt/token/encryption-key | Go route recorded: True

- POST /consumerauthorization/authorization/mgmt/token/generate | Go route recorded: True

- POST /consumerauthorization/authorization/mgmt/token/query | Go route recorded: True

- DELETE /consumerauthorization/authorization/mgmt/token/revoke | Go route recorded: True

- DELETE /consumerauthorization/authorization/revoke/{instanceId} | Go route recorded: True

- POST /consumerauthorization/authorization/verify | Go route recorded: True

- GET /consumerauthorization/general/mgmt/get-config | Go route recorded: True

- POST /consumerauthorization/general/mgmt/logs | Go route recorded: True

- GET /consumerauthorization/monitor/echo | Go route recorded: True

Native compatibility profile

Compatibility profile and remaining work

This document is deliberately separate from the route inventory. It records the actual implemented behavior and material gaps, rather than calling a set of HTTP handlers a complete Java replacement.

Implemented

- Authentication: PASSWORD identities, bcrypt cost 10, batch management, credential changes, credential-authenticated logout, UUID sessions, one active session per identity, expiration checks, session query/closure and identity query filters. Credentials and session tokens are excluded from management responses. Credential changes and management updates invalidate sessions (a stricter security choice).

- ConsumerAuthorization: provider and MGMT policy levels, MGMT precedence, ALL/WHITELIST/BLACKLIST/SYS_METADATA policies, scoped/default fallback, all-scopes checks, grant/lookup/query/revoke/check operations. Missing policies deny access.

- Authorization tokens: usage/time-limited tokens; URL-base64 self-contained tokens with the Java pipe-separated field order; RS256/RS512 JWTs with psn/csn/ccn/tat/tan/sco claims; public-key export; management generation/query/revocation; provider-bound online verification with atomic usage decrement. Raw tokens are returned only at generation, not retained for later retrieval.

- Encryption-key registration: Java AES/ECB/PKCS5Padding and AES/CBC/PKCS5Padding wire encodings, provider-specific IVs for CBC, owner/management removal. Provider keys are authenticated-encrypted at rest with AES-GCM and provider identity as additional authenticated data. The server certificate's RSA key signs JWTs.

- Simple orchestration: consumer-specific store, unique priorities per consumer/service, priority modification, preferred providers, ONLY_PREFERRED, per-service MATCHMAKING, ignored-field/flag warnings, pull/history.

- Dynamic orchestration: service discovery through ServiceRegistry, mandatory authorization checks, preferred providers, interface operations, token acquisition, MATCHMAKING, provider-consented exclusivity capped by allowExclusivity, atomic lock conflict rechecks, lock management, pull/history.

- Both orchestrators: persisted subscriptions, owner checks, management subscription/query/unsubscribe/trigger operations, persisted jobs and HTTPS push callbacks. Callback TLS is pinned to the target system identity before sending results. Interrupted in-progress jobs become ERROR on restart instead of being redelivered automatically.

- General management: sanitized configuration map, bounded persisted audit log querying, authenticated echo. Source-generated service descriptions and opt-in periodic service registration.

- Shared runtime: mandatory local-cloud mTLS, identity-token certificate binding, fixed authority identity pinning before credentials, redirect rejection, coalesced login refresh, conditional token invalidation, body limits, server timeouts, atomic JSON snapshots and exclusive process ownership of a state file.

- ServiceRegistry: pre-existing Go implementation plus corrected AuthorizationVerifyRequestDTO fields/operation names and removal of an implicit Sysop-name privilege escalation in outsourced token verification. Its original configuration and publisher are preserved.

Not implemented / not equivalent

1. MQTT APIs (since 2026-10-24) differ from Java in three deliberate points: the response topic must lie under arrowhead/response/<requester>/, requests with an invalid identity token are not answered (no 401 response), and identity login/logout/change stay HTTPS only. Request topics, envelopes, operation names and error payloads follow the Java MQTT API; see "Changes 2026-10-24 (MQTT)". 2. Dynamic federation, translation and QoS integration are absent. Requests that enable ALLOW_INTERCLOUD, ONLY_INTERCLOUD or ALLOW_TRANSLATION, or supply QoS requirements, fail explicitly with 501. They are not silently treated as successful local equivalents. Foreign-consumer management token generation also returns 501. This is unsuitable for a federated orchestration replacement until those paths are ported and tested. 3. Blacklist support and authorization-driven management filters are not ported to the four new systems. Management uses identity sysop privileges or an explicit whole-management whitelist. This is less granular than per-operation authorization; do not use the whitelist as a substitute for fine-grained policy enforcement. 4. The new systems do not yet publish native sysMonitor/AMQP telemetry. Their echo APIs and history/logs exist, but they will not satisfy a LocalCloudManager native-telemetry readiness requirement. The existing ServiceRegistry publisher remains available. 5. No Java SQL database compatibility/import. Each new system uses a private atomic JSON state file, not JPA/MySQL. Never mount a Java database volume as a Go state file. Java identity, authorization, key, store, lock, subscription and history data need a separately reviewed importer before migration. 6. Configuration is JSON, not Spring properties, environment override semantics, keystores or Java's full property catalog. get-config returns a sanitized native configuration subset in the Java map response wrapper. There is no cloud registration health/readiness gate; registration failures are retried and reported to the process log. 7. Validation/normalization and query semantics are not exhaustively matched. Supply canonical names and enum spelling. Field lengths, case normalization, HTTP error details/statuses, empty/null distinctions, ordering/default page sizes and warning text still need Java golden-contract tests. Pagination is bounded, but is not a full port of Java's per-entity paging service. JSON inputs use dynamic documents rather than generated, exhaustive DTO validation. 8. Metadata supports the documented comparisons and dotted/indexed paths, but REGEXP uses Go RE2, not Java Pattern; lookarounds/backreferences are not supported. Mixed-type numeric/collection edge cases need further Java fixture tests. 9. Dynamic candidate ranking is deterministic local/preferred ordering, not the complete Java matchmaking/QoS/fallback pipeline. Scopes are checked fail-closed; a denied requested operation can reject orchestration instead of yielding Java's precise partial-result behavior. Token acquisition is not rolled back if a later lock conflict aborts orchestration. 10. Push uses one worker and one delivery attempt, without Java's Quartz/thread-pool behavior, HTTP/plain MQTT callbacks, retries, or automatic event-driven re-orchestration. HTTPS result-delivery behavior is tested; duplicate delivery avoidance does not constitute exactly-once delivery. Notifications interrupted by a crash are reported as uncertain ERROR. 11. Logs are native bounded audit records (1,000 retained), not the full Java logging database or JVM-specific exceptions. Sessions/tokens/jobs/subscriptions/locks are validated for expiry at access but do not have complete Java cleaner/retention policies; state can grow over time. Expired entities may differ in query visibility. 12. No certificate revocation enforcement beyond Go's ordinary certificate-chain/time/hostname checks. No blacklist, rate limiter, distributed transactions, HA, distributed locks, SQL migration, or production security certification. Offline self-contained token revocation requires provider-side expiration/revocation handling; online revocation does not invalidate already-issued offline tokens.

Release gate

Do not describe this profile as fully interchangeable with the Java cloud. Before replacement: implement the required missing capabilities, compare successful and failing requests against Java for every DTO/operation, verify all discovery/security policies, migrate data with rollback, verify monitoring/UI readiness and federation, and rerun the cloud security evaluation. None of those deployment changes is made by this source delivery.

Changes 2026-10-17 (GoCloud deployment)

- Dynamic orchestration passes each requested operation as an authorization scope; a provider qualifies only if

every requested operation is granted. Without operations the all-scopes check applies (as before). Test: TestNativeServiceRegistryIntegration.

- Deployed in Arrowhead-test-cloud-go (see its GO-NATIVE-FULL-2026-10-17.md) by operator decision, with the gaps

listed above accepted; the release gate above is not thereby satisfied.

- Inter-cloud orchestration: ALLOW_INTERCLOUD (fallback when no local provider) and ONLY_INTERCLOUD through the local

Gatekeeper (gatekeeperURL): globalServiceDiscovery query, interCloudNegotiation negotiate, GateTunnel interface result. ALLOW_TRANSLATION and QoS requirements still return 501.

- Native sysMonitor telemetry (monitorEnabled, amqpURL): sysMonitor offer on generic_amqps, echo/power/status.

- managementPolicy: authorization: ConsumerAuthorization decides management access per service and operation.

- Requested operations match string-set operations (MQTT/AMQP templates) as well as HTTP operation maps.

- Push subscriptions store notifyRecipient: a non-sysop management subscriber (orchestration by proxy) receives

the result; own and sysop subscriptions deliver to the target.

Changes 2026-10-24 (translation)

- ALLOW_TRANSLATION: an interface is translatable when it does not match the template or property requirements

natively (as Java assortInterfacesAndMarkIfNonNative), so a bridge can also translate only the data model of the accepted template (generic_https senml -> generic_https simpleJson). Native candidates still win; the consumer's authorization for the provider is still checked before a bridge is built.

Changes 2026-10-24 (MQTT)

- MQTT API of all five core systems (internal/core/mqtt.go, ServiceRegistry/mqtt.go), the Java mqtt.api.enabled

counterpart: the Java request envelope (MqttRequestTemplate: traceId, authentication IDENTITY-TOKEN//<token>, responseTopic, qosRequirement, params, payload) on the Java base topics + operation (arrowhead/serviceregistry/{device-discovery,system-discovery,service-discovery,management,general/management, monitor}/, arrowhead/authentication/identity/[management/], arrowhead/consumer-authorization/authorization[-token]/ [management/], arrowhead/serviceorchestration/orchestration/[management/{push,history,lock,store,plant-rules}/], general/management/, monitor/echo); the response (MqttResponseTemplate: status, traceId, receiver, payload; errors as ErrorMessageDTO with exceptionType and origin = topic) goes to the request's response topic.

- The adapter verifies the identity token and runs the request in process through the HTTPS route, so the blacklist,

management (whitelist/authorization) checks and handlers are the same as over HTTPS. Path parameters come from params (a single one also from a string payload, as Java), the other params become the query, a list payload of a removal operation is its list parameter (names, instanceIds, uuids, ids, tokenReferences, systemNames, serviceInstances, keys).

- Response binding (Go): the response topic must lie under arrowhead/response/<system of the token>/; with a broker

ACL that lets each client read only its own prefix, a stolen token cannot be used to read answers.

- Configuration: mqttURL (ssl://host:port, client certificate = system certificate) and mqttTopicPrefix (a second

orchestrator on the same broker needs another prefix); ServiceRegistry mqtt.api.enabled, mqtt.broker.address, mqtt.broker.port, mqtt.topic.prefix (requires TLS and the outsourced policy). With MQTT enabled every service is

also registered with a generic_mqtts interface (baseTopic, operation list), as Java.

- ServiceRegistry: the Blacklist filter never checks Blacklist itself (Java BlacklistFilter); a Java Blacklist, which

registers before it serves its check, was refused under force.blacklist.filter.

- cmd/mqtt-call: command-line client for one MQTT API request (tests).

Rust implementation

Primary folder: Eclipse-Arrowhead-rust/ah5-core-rust/consumer-authorization.

ConsumerAuthorization (Rust)

Arrowhead 5.2.2 core system. src/main.rs is built as binary consumer-authorization by the arrowhead-core-rust crate in ... consumer-authorization.json and documentation/ link to ../examples and ../documentation/ConsumerAuthorization.

Detailed implementation evidence

1 native Rust IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Build and state ownership

Core entry points are built by arrowhead-core-rust; the ordinary support entry points are built by arrowhead-support-rust using the shared support runtime. LocalCloudManager has its own crate. Per-system documentation links may point into the shared documentation catalog. Independent identity and state configuration is still required. Read the native appendix for persistence and restart semantics.

Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data migration, deployment or cloud reconfiguration was performed for this document set.

Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-authorization-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-authorization-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-authorizationManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-authorizationManagement-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-authorizationToken-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-authorizationToken-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-authorizationTokenManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-authorizationTokenManagement-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-generalManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-generalManagement-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-sysMonitor-AMQP-IDD.tex

- Rust: Eclipse-Arrowhead-rust/ah5-core-rust/documentation/ConsumerAuthorization/ConsumerAuthorization-HTTPS-IDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

authorization HTTP/TLS/JSON

Interface Design Description

Service ID: *"authorization"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `authorization` service of the ConsumerAuthorization system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation grant . . . 4
2.2 operation revoke . . . 5
2.3 operation lookup . . . 5
2.4 operation verify . . . 6

3 Data Models **8**
3.1 struct AuthorizationGrantRequest . . . 8
3.2 struct AuthorizationLookupRequest . . . 8
3.3 struct AuthorizationPolicy . . . 8
3.4 struct AuthorizationPolicyListResponse . . . 8
3.5 struct AuthorizationPolicyRequest . . . 9
3.6 struct AuthorizationPolicyResponse . . . 9
3.7 struct AuthorizationVerifyRequest . . . 9
3.8 struct ErrorMessage . . . 10
3.9 struct MetadataRequirement . . . 10
3.10 Primitives . . . 10

4 References **11**

5 Revision History **12**
5.1 Amendments . . . 12
5.2 Quality Assurance . . . 12

1 Overview

This document describes the `authorization` service interface of the ConsumerAuthorization system, which provides provider-level authorization policies: a provider declares which consumers may use its service (or event type), and any system can ask whether a consumer is authorized.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `authorization` service interface

This document provides the Interface Design Description IDD to the *authorization – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `authorization` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8445 and base path `/consumerauthorization/authorization`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`grant`	POST	`/grant`
`revoke`	DELETE	`/revoke/{instanceId}`
`lookup`	POST	`/lookup`
`verify`	POST	`/verify`

2 Service Operations

The interfaces of the `authorization` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `authorization` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /consumerauthorization/authorization/grant

Operation: grant
Input: AuthorizationGrantRequest
Output: AuthorizationPolicyResponse

Creates or replaces the provider-level policy of the requesting provider for one target. Returns 201 for a new policy, 200 if an identical one exists.

A successful call returns `201 Created` for a new entry, `200 OK` if an identical entry already existed with the output in the body.

```
POST /consumerauthorization/authorization/grant HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "cloud": "LOCAL",
  "targetType": "SERVICE_DEF",
  "target": "temperatureReading",
  "description": "Displays may read the temperature",
  "defaultPolicy": {
    "policyType": "WHITELIST",
    "policyList": [
      "TemperatureDisplay"
    ],
    "policyMetadataRequirement": {
      "unit": "celsius"
    }
  },
  "scopedPolicies": {
    "read": {
      "policyType": "ALL"
    }
  }
}
```

Listing 1: An grant invocation.

```
{
  "instanceId": "TemperatureSensor|temperatureReading|1.0.0",
  "level": "MGMT",
  "cloud": "LOCAL",
  "provider": "TemperatureSensor",
  "targetType": "SERVICE_DEF",
```

```
  "target": "temperatureReading",
  "description": "Display may read",
  "defaultPolicy": {
    "policyType": "ALL",
    "policyList": [
      "TemperatureDisplay"
    ],
    "policyMetadataRequirement": {
      "unit": "celsius"
    }
  },
  "scopedPolicies": {
    "read": {
      "policyType": "ALL"
    }
  },
  "createdBy": "Sysop",
  "createdAt": "2026-10-22T12:00:00Z"
}
```

Listing 2: An grant response.

2.2 DELETE /consumerauthorization/authorization/revoke/*{instanceId}*

Operation: revoke
Input: String

Removes a provider-level policy of the requester by instance identifier. Returns 200 if removed, 204 if it did not exist.

Path parameter: *instanceId*.

A successful call returns `200 OK` if the entry was removed, `204 No Content` if there was nothing to remove.

```
DELETE /consumerauthorization/authorization/revoke/<instanceId> HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An revoke invocation.

2.3 POST /consumerauthorization/authorization/lookup

Operation: lookup
Input: AuthorizationLookupRequest
Output: AuthorizationPolicyListResponse

Returns the requester's provider-level policies matching the filter.

A successful call returns `200 OK` with the output in the body.

```
POST /consumerauthorization/authorization/lookup HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "instanceIds": [
    "TemperatureSensor|temperatureReading|1.0.0"
  ],
  "cloudIdentifiers": [
    "string"
  ],
  "targetNames": [
```

```
    "string"
  ],
  "targetType": "SERVICE_DEF"
}
```

Listing 4: An lookup invocation.

```
{
  "entries": [
    {
      "instanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "level": "MGMT",
      "cloud": "LOCAL",
      "provider": "TemperatureSensor",
      "targetType": "SERVICE_DEF",
      "target": "temperatureReading",
      "description": "Display may read",
      "defaultPolicy": {
        "policyType": "ALL",
        "policyList": [
          "TemperatureDisplay"
        ],
        "policyMetadataRequirement": {
          "unit": "celsius"
        }
      },
      "scopedPolicies": {
        "read": {
          "policyType": "ALL"
        }
      },
      "createdBy": "Sysop",
      "createdAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 5: An lookup response.

2.4 POST /consumerauthorization/authorization/verify

Operation: verify
Input: AuthorizationVerifyRequest
Output: Boolean

Evaluates whether the consumer may use the target (and operation) of the provider: management-level policies first, provider-level policies if none applies; no applicable policy means denied. Returns a boolean.

A successful call returns `200 OK` with the output in the body.

```
POST /consumerauthorization/authorization/verify HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "provider": "TemperatureSensor",
  "consumer": "TemperatureDisplay",
  "cloud": "LOCAL",
  "targetType": "SERVICE_DEF",
  "target": "temperatureReading",
  "scope": "read"
}
```

Listing 6: An verify invocation.

```
true
```

Listing 7: An verify response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.10, which are used to represent things like hashes, identifiers and texts.

3.1 struct AuthorizationGrantRequest

A provider-level authorization policy that the provider sets for its own service.

Object Field	Value Type	Description
"cloud"	String	Cloud identifier of the consumer cloud the policy applies to; `LOCAL` for the own cloud.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Service definition (or event type) the policy protects.
"description"	String	Free-text description.
"defaultPolicy"	AuthorizationPolicyRequest	Policy that applies to all operations without a scoped policy.
"scopedPolicies"	Object<AuthorizationPolicyRequest>	Policies for individual operations, keyed by operation name.

3.2 struct AuthorizationLookupRequest

Filter for the requester's own provider-level policies.

Object Field	Value Type	Description
"instanceIds"	Array<String>	Policy instance identifiers to match.
"cloudIdentifiers"	Array<String>	Cloud identifiers to match.
"targetNames"	Array<String>	Target names to match.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.

3.3 struct AuthorizationPolicy

One policy rule.

Object Field	Value Type	Description
"policyType"	AuthorizationPolicyType	Policy type: `ALL`, `WHITELIST`, `BLACKLIST` or `SYS_METADATA`.
"policyList"	Array<String>	System names of the white- or blacklist.
"policyMetadata Requirement"	MetadataRequirement	Metadata the consumer system must have (`SYS_METADATA`).

3.4 struct AuthorizationPolicyListResponse

List of authorization policies.

Object Field	Value Type	Description
"entries"	Array<AuthorizationPolicy Response>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.5 struct AuthorizationPolicyRequest

One policy rule.

Object Field	Value Type	Description
"policyType"	String	Policy type: `ALL`, `WHITELIST`, `BLACKLIST` or `SYS_METADATA`.
"policyList"	Array<String>	System names of the white- or blacklist.
"policyMetadata Requirement"	MetadataRequirement	Metadata the consumer system must have (`SYS_METADATA`).

3.6 struct AuthorizationPolicyResponse

A stored authorization policy.

Object Field	Value Type	Description
"instanceId"	String	Policy instance identifier `<MGMT or PR>|<cloud>|<provider>|<targetType>|<target>`.
"level"	AuthorizationLevel	Policy level: `MGMT` (set by an operator) or `PROVIDER` (set by the provider itself).
"cloud"	String	Cloud identifier of the consumer cloud the policy applies to; `LOCAL` for the own cloud.
"provider"	String	Provider system the policy protects.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"description"	String	Free-text description.
"defaultPolicy"	AuthorizationPolicy	Policy that applies to all operations without a scoped policy.
"scopedPolicies"	Object<AuthorizationPolicy>	Policies for individual operations, keyed by operation name.
"createdBy"	String	Name of the system that created the record.
"createdAt"	DateTime	UTC time of creation.

3.7 struct AuthorizationVerifyRequest

One authorization question: may the consumer use the target (operation) of the provider?

Object Field	Value Type	Description
"provider"	String	Provider system; for `verify` defaults to the requester.
"consumer"	String	Consumer system; for `verify` defaults to the requester.
"cloud"	String	Cloud of the consumer, `LOCAL` by default.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"scope"	String	Operation to check; empty checks the default policy.

3.8 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.9 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.10 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
AuthorizationLevel	One of `MGMT`, `PROVIDER` (JSON String).
AuthorizationPolicyType	One of `ALL`, `WHITELIST`, `BLACKLIST`, `SYS_METADATA` (JSON String).
AuthorizationTargetType	One of `SERVICE_DEF`, `EVENT_TYPE` (JSON String).
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “authorization – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorization-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

authorization MQTT/TLS/JSON

Interface Design Description

Service ID: *"authorization"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `authorization` service of the ConsumerAuthorization system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation grant . . . 4
2.2 operation revoke . . . 5
2.3 operation lookup . . . 5
2.4 operation verify . . . 5

3 Data Models **7**
3.1 struct MqttRequestTemplate . . . 7
3.2 struct MqttResponseTemplate . . . 7
3.3 struct ErrorMessage . . . 7
3.4 struct AuthorizationGrantRequest . . . 7
3.5 struct AuthorizationLookupRequest . . . 8
3.6 struct AuthorizationPolicy . . . 8
3.7 struct AuthorizationPolicyListResponse . . . 8
3.8 struct AuthorizationPolicyRequest . . . 8
3.9 struct AuthorizationPolicyResponse . . . 9
3.10 struct AuthorizationVerifyRequest . . . 9
3.11 struct MetadataRequirement . . . 9
3.12 Primitives . . . 10

4 References **11**

5 Revision History **12**
5.1 Amendments . . . 12
5.2 Quality Assurance . . . 12

1 Overview

This document describes the `authorization` service interface of the ConsumerAuthorization system over MQTT, which provides provider-level authorization policies: a provider declares which consumers may use its service (or event type), and any system can ask whether a consumer is authorized.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `authorization` service interface

This document provides the Interface Design Description IDD to the *authorization – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/consumer-authorization/authorization/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`grant`	`arrowhead/consumer-authorization/authorization/grant`
`revoke`	`arrowhead/consumer-authorization/authorization/revoke`
`lookup`	`arrowhead/consumer-authorization/authorization/lookup`
`verify`	`arrowhead/consumer-authorization/authorization/verify`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/consumer-authorization/authorization/grant`

Operation: grant
Input: AuthorizationGrantRequest
Output: AuthorizationPolicyResponse

Creates or replaces the provider-level policy of the requesting provider for one target. Returns 201 for a new policy, 200 if an identical one exists.

```
{
  "cloud": "LOCAL",
  "targetType": "SERVICE_DEF",
  "target": "temperatureReading",
  "description": "Displays may read the temperature",
  "defaultPolicy": {
    "policyType": "WHITELIST",
    "policyList": [
      "TemperatureDisplay"
    ],
    "policyMetadataRequirement": {
      "unit": "celsius"
```

```
    }
  },
  "scopedPolicies": {
    "read": {
      "policyType": "ALL"
    }
  }
}
```

Listing 3: Payload of an grant request.

2.2 PUBLISH arrowhead/consumer-authorization/authorization/revoke

Operation: revoke
Input: String

Removes a provider-level policy of the requester by instance identifier. Returns 200 if removed, 204 if it did not exist.

```
"TemperatureSensor|temperatureReading|1.0.0"
```

Listing 4: Payload of an revoke request.

2.3 PUBLISH arrowhead/consumer-authorization/authorization/lookup

Operation: lookup
Input: AuthorizationLookupRequest
Output: AuthorizationPolicyListResponse

Returns the requester's provider-level policies matching the filter.

```
{
  "instanceIds": [
    "TemperatureSensor|temperatureReading|1.0.0"
  ],
  "cloudIdentifiers": [
    "string"
  ],
  "targetNames": [
    "string"
  ],
  "targetType": "SERVICE_DEF"
}
```

Listing 5: Payload of an lookup request.

2.4 PUBLISH arrowhead/consumer-authorization/authorization/verify

Operation: verify
Input: AuthorizationVerifyRequest
Output: Boolean

Evaluates whether the consumer may use the target (and operation) of the provider: management-level policies first, provider-level policies if none applies; no applicable policy means denied. Returns a boolean.

```
{
  "provider": "TemperatureSensor",
  "consumer": "TemperatureDisplay",
  "cloud": "LOCAL",
```

```
  "targetType": "SERVICE_DEF",
  "target": "temperatureReading",
  "scope": "read"
}
```

Listing 6: Payload of an verify request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct AuthorizationGrantRequest

A provider-level authorization policy that the provider sets for its own service.

Object Field	Value Type	Description
"cloud"	String	Cloud identifier of the consumer cloud the policy applies to; `LOCAL` for the own cloud.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Service definition (or event type) the policy protects.
"description"	String	Free-text description.
"defaultPolicy"	AuthorizationPolicyRequest	Policy that applies to all operations without a scoped policy.
"scopedPolicies"	Object<AuthorizationPolicyRequest>	Policies for individual operations, keyed by operation name.

3.5 struct AuthorizationLookupRequest

Filter for the requester's own provider-level policies.

Object Field	Value Type	Description
"instanceIds"	Array<String>	Policy instance identifiers to match.
"cloudIdentifiers"	Array<String>	Cloud identifiers to match.
"targetNames"	Array<String>	Target names to match.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.

3.6 struct AuthorizationPolicy

One policy rule.

Object Field	Value Type	Description
"policyType"	AuthorizationPolicyType	Policy type: `ALL`, `WHITELIST`, `BLACKLIST` or `SYS_METADATA`.
"policyList"	Array<String>	System names of the white- or blacklist.
"policyMetadata Requirement"	MetadataRequirement	Metadata the consumer system must have (`SYS_METADATA`).

3.7 struct AuthorizationPolicyListResponse

List of authorization policies.

Object Field	Value Type	Description
"entries"	Array<AuthorizationPolicy Response>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.8 struct AuthorizationPolicyRequest

One policy rule.

Object Field	Value Type	Description
"policyType"	String	Policy type: `ALL`, `WHITELIST`, `BLACKLIST` or `SYS_METADATA`.
"policyList"	Array<String>	System names of the white- or blacklist.
"policyMetadata Requirement"	MetadataRequirement	Metadata the consumer system must have (`SYS_METADATA`).

3.9 struct AuthorizationPolicyResponse

A stored authorization policy.

Object Field	Value Type	Description
"instanceId"	String	Policy instance identifier `<MGMT or PR>|<cloud>|<provider>|<targetType>|<target>`.
"level"	AuthorizationLevel	Policy level: `MGMT` (set by an operator) or `PROVIDER` (set by the provider itself).
"cloud"	String	Cloud identifier of the consumer cloud the policy applies to; `LOCAL` for the own cloud.
"provider"	String	Provider system the policy protects.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"description"	String	Free-text description.
"defaultPolicy"	AuthorizationPolicy	Policy that applies to all operations without a scoped policy.
"scopedPolicies"	Object<AuthorizationPolicy>	Policies for individual operations, keyed by operation name.
"createdBy"	String	Name of the system that created the record.
"createdAt"	DateTime	UTC time of creation.

3.10 struct AuthorizationVerifyRequest

One authorization question: may the consumer use the target (operation) of the provider?

Object Field	Value Type	Description
"provider"	String	Provider system; for `verify` defaults to the requester.
"consumer"	String	Consumer system; for `verify` defaults to the requester.
"cloud"	String	Cloud of the consumer, `LOCAL` by default.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"scope"	String	Operation to check; empty checks the default policy.

3.11 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.12 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
AuthorizationLevel	One of `MGMT`, `PROVIDER` (JSON String).
AuthorizationPolicyType	One of `ALL`, `WHITELIST`, `BLACKLIST`, `SYS_METADATA` (JSON String).
AuthorizationTargetType	One of `SERVICE_DEF`, `EVENT_TYPE` (JSON String).
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “authorization – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorization-SD.pdf.

[6] ——, “authorization HTTP/TLS/JSON – Interface Design Description (ConsumerAuthorization),” Eclipse Arrowhead, IDD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorization-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

authorizationManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"authorizationManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `authorizationManagement` service of the ConsumerAuthorization system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation grant-policies . . . 4
2.2 operation revoke-policies . . . 5
2.3 operation query-policies . . . 5
2.4 operation check-policies . . . 6

3 Data Models **8**
3.1 struct AuthorizationMgmtGrantListRequest . . . 8
3.2 struct AuthorizationMgmtGrantRequest . . . 8
3.3 struct AuthorizationPolicy . . . 8
3.4 struct AuthorizationPolicyListResponse . . . 8
3.5 struct AuthorizationPolicyRequest . . . 8
3.6 struct AuthorizationPolicyResponse . . . 9
3.7 struct AuthorizationQueryRequest . . . 9
3.8 struct AuthorizationVerifyListRequest . . . 9
3.9 struct AuthorizationVerifyListResponse . . . 10
3.10 struct AuthorizationVerifyRequest . . . 10
3.11 struct AuthorizationVerifyResponse . . . 10
3.12 struct ErrorMessage . . . 10
3.13 struct MetadataRequirement . . . 11
3.14 struct Page . . . 11
3.15 Primitives . . . 11

4 References **12**

5 Revision History **13**
5.1 Amendments . . . 13
5.2 Quality Assurance . . . 13

1 Overview

This document describes the `authorizationManagement` service interface of the ConsumerAuthorization system, which provides operator management of authorization policies at the management level, and bulk evaluation of authorization questions.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `authorizationManagement` service interface

This document provides the Interface Design Description IDD to the *authorizationManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `authorizationManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8445 and base path `/consumerauthorization/authorization/mgmt`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`grant-policies`	POST	`/grant`
`revoke-policies`	DELETE	`/revoke`
`query-policies`	POST	`/query`
`check-policies`	POST	`/check`

2 Service Operations

The interfaces of the `authorizationManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `authorizationManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /consumerauthorization/authorization/mgmt/grant

Operation: grant-policies
Input: AuthorizationMgmtGrantListRequest
Output: AuthorizationPolicyListResponse

Creates management-level policies for the given providers and targets.

A successful call returns `201 Created` with the output in the body.

```
POST /consumerauthorization/authorization/mgmt/grant HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "list": [
    {
      "cloud": "LOCAL",
      "provider": "TemperatureSensor",
      "targetType": "SERVICE_DEF",
      "target": "temperatureReading",
      "description": "Display may read",
      "defaultPolicy": {
        "policyType": "WHITELIST",
        "policyList": [
          "TemperatureDisplay"
        ],
        "policyMetadataRequirement": {
          "unit": "celsius"
        }
      },
      "scopedPolicies": {
        "read": {
          "policyType": "ALL"
        }
      }
    }
  ]
}
```

Listing 1: An grant-policies invocation.

```
{
  "entries": [
    {
```

```
      "instanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "level": "MGMT",
      "cloud": "LOCAL",
      "provider": "TemperatureSensor",
      "targetType": "SERVICE_DEF",
      "target": "temperatureReading",
      "description": "Display may read",
      "defaultPolicy": {
        "policyType": "ALL",
        "policyList": [
          "TemperatureDisplay"
        ],
        "policyMetadataRequirement": {
          "unit": "celsius"
        }
      },
      "scopedPolicies": {
        "read": {
          "policyType": "ALL"
        }
      },
      "createdBy": "Sysop",
      "createdAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 2: An grant-policies response.

2.2 DELETE /consumerauthorization/authorization/mgmt/revoke?instanceIds=*{instanceIds}*

Operation: revoke-policies
Input: Array<String>

Removes policies by instance identifier.
Query parameters: *instanceIds* (List<String>, required). List values are repeated or comma-separated.
A successful call returns `200 OK`.

```
DELETE /consumerauthorization/authorization/mgmt/revoke?instanceIds=<instanceIds> HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An revoke-policies invocation.

2.3 POST /consumerauthorization/authorization/mgmt/query

Operation: query-policies
Input: AuthorizationQueryRequest
Output: AuthorizationPolicyListResponse

Returns a page of policies of either level matching the filter.
A successful call returns `200 OK` with the output in the body.

```
POST /consumerauthorization/authorization/mgmt/query HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
```

```
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "level": "MGMT",
  "providers": [
    "TemperatureSensor"
  ],
  "instanceIds": [
    "TemperatureSensor|temperatureReading|1.0.0"
  ],
  "cloudIdentifiers": [
    "string"
  ],
  "targetNames": [
    "string"
  ],
  "targetType": "SERVICE_DEF"
}
```

Listing 4: An query-policies invocation.

```
{
  "entries": [
    {
      "instanceId": "TemperatureSensor|temperatureReading|1.0.0",
      "level": "MGMT",
      "cloud": "LOCAL",
      "provider": "TemperatureSensor",
      "targetType": "SERVICE_DEF",
      "target": "temperatureReading",
      "description": "Display may read",
      "defaultPolicy": {
        "policyType": "ALL",
        "policyList": [
          "TemperatureDisplay"
        ],
        "policyMetadataRequirement": {
          "unit": "celsius"
        }
      },
      "scopedPolicies": {
        "read": {
          "policyType": "ALL"
        }
      },
      "createdBy": "Sysop",
      "createdAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 5: An query-policies response.

2.4 POST /consumerauthorization/authorization/mgmt/check

Operation: check-policies
Input: AuthorizationVerifyListRequest
Output: AuthorizationVerifyListResponse

Evaluates a list of authorization questions and returns the answer for each.
A successful call returns `200 OK` with the output in the body.

```
POST /consumerauthorization/authorization/mgmt/check HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "list": [
    {
      "provider": "TemperatureSensor",
      "consumer": "TemperatureDisplay",
      "cloud": "LOCAL",
      "targetType": "SERVICE_DEF",
      "target": "temperatureReading",
      "scope": "read"
    }
  ]
}
```

Listing 6: An check-policies invocation.

```
{
  "entries": [
    {
      "provider": "TemperatureSensor",
      "consumer": "TemperatureDisplay",
      "cloud": "LOCAL",
      "targetType": "SERVICE_DEF",
      "target": "temperatureReading",
      "scope": "read",
      "granted": true
    }
  ],
  "count": 1
}
```

Listing 7: An check-policies response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.15, which are used to represent things like hashes, identifiers and texts.

3.1 struct AuthorizationMgmtGrantListRequest

Management-level policies to create.

Object Field	Value Type	Description
"list"	Array<AuthorizationMgmtGrant Request>	The list of request items.

3.2 struct AuthorizationMgmtGrantRequest

A management-level authorization policy for one provider and target.

Object Field	Value Type	Description
"cloud"	String	Cloud identifier of the consumer cloud the policy applies to; `LOCAL` for the own cloud.
"provider"	String	Provider system the policy is set for.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"description"	String	Free-text description.
"defaultPolicy"	AuthorizationPolicyRequest	Policy that applies to all operations without a scoped policy.
"scopedPolicies"	Object<AuthorizationPolicyRequest>	Policies for individual operations, keyed by operation name.

3.3 struct AuthorizationPolicy

One policy rule.

Object Field	Value Type	Description
"policyType"	AuthorizationPolicyType	Policy type: `ALL`, `WHITELIST`, `BLACKLIST` or `SYS_METADATA`.
"policyList"	Array<String>	System names of the white- or blacklist.
"policyMetadata Requirement"	MetadataRequirement	Metadata the consumer system must have (`SYS_METADATA`).

3.4 struct AuthorizationPolicyListResponse

List of authorization policies.

Object Field	Value Type	Description
"entries"	Array<AuthorizationPolicy Response>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.5 struct AuthorizationPolicyRequest

One policy rule.

Object Field	Value Type	Description
"policyType"	String	Policy type: `ALL`, `WHITELIST`, `BLACKLIST` or `SYS_METADATA`.
"policyList"	Array<String>	System names of the white- or blacklist.
"policyMetadata Requirement"	MetadataRequirement	Metadata the consumer system must have (`SYS_METADATA`).

3.6 struct AuthorizationPolicyResponse

A stored authorization policy.

Object Field	Value Type	Description
"instanceId"	String	Policy instance identifier `<MGMT or PR>|<cloud>|<provider>|<targetType>|<target>`.
"level"	AuthorizationLevel	Policy level: `MGMT` (set by an operator) or `PROVIDER` (set by the provider itself).
"cloud"	String	Cloud identifier of the consumer cloud the policy applies to; `LOCAL` for the own cloud.
"provider"	String	Provider system the policy protects.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"description"	String	Free-text description.
"defaultPolicy"	AuthorizationPolicy	Policy that applies to all operations without a scoped policy.
"scopedPolicies"	Object<AuthorizationPolicy>	Policies for individual operations, keyed by operation name.
"createdBy"	String	Name of the system that created the record.
"createdAt"	DateTime	UTC time of creation.

3.7 struct AuthorizationQueryRequest

Filter for authorization policies.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"level"	String	Policy level to match, `MGMT` or `PROVIDER`.
"providers"	Array<String>	Provider system names to match.
"instanceIds"	Array<String>	Policy instance identifiers to match.
"cloudIdentifiers"	Array<String>	Cloud identifiers to match.
"targetNames"	Array<String>	Target names to match.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.

3.8 struct AuthorizationVerifyListRequest

Authorization checks to evaluate.

Object Field	Value Type	Description
"list"	Array<AuthorizationVerifyRequest>	The list of request items.

3.9 struct AuthorizationVerifyListResponse

Results of authorization checks.

Object Field	Value Type	Description
"entries"	Array<AuthorizationVerifyResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.10 struct AuthorizationVerifyRequest

One authorization question: may the consumer use the target (operation) of the provider?

Object Field	Value Type	Description
"provider"	String	Provider system; for `verify` defaults to the requester.
"consumer"	String	Consumer system; for `verify` defaults to the requester.
"cloud"	String	Cloud of the consumer, `LOCAL` by default.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"scope"	String	Operation to check; empty checks the default policy.

3.11 struct AuthorizationVerifyResponse

The answer to one authorization question.

Object Field	Value Type	Description
"provider"	String	Name of the provider system.
"consumer"	String	Name of the consumer system.
"cloud"	String	Cloud of the consumer.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"scope"	String	The checked operation.
"granted"	Boolean	Result of the check: true when the consumer is authorized.

3.12 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.13 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.14 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.15 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
AuthorizationLevel	One of `MGMT`, `PROVIDER` (JSON String).
AuthorizationPolicyType	One of `ALL`, `WHITELIST`, `BLACKLIST`, `SYS_METADATA` (JSON String).
AuthorizationTargetType	One of `SERVICE_DEF`, `EVENT_TYPE` (JSON String).
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “authorizationManagement – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorizationManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

authorizationManagement MQTT/TLS/JSON

Interface Design Description

Service ID: *"authorizationManagement"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `authorizationManagement` service of the ConsumerAuthorization system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation grant-policies . . . 4
2.2 operation revoke-policies . . . 5
2.3 operation query-policies . . . 5
2.4 operation check-policies . . . 6

3 Data Models **7**
3.1 struct MqttRequestTemplate . . . 7
3.2 struct MqttResponseTemplate . . . 7
3.3 struct ErrorMessage . . . 7
3.4 struct AuthorizationMgmtGrantListRequest . . . 7
3.5 struct AuthorizationMgmtGrantRequest . . . 7
3.6 struct AuthorizationPolicy . . . 8
3.7 struct AuthorizationPolicyListResponse . . . 8
3.8 struct AuthorizationPolicyRequest . . . 8
3.9 struct AuthorizationPolicyResponse . . . 8
3.10 struct AuthorizationQueryRequest . . . 9
3.11 struct AuthorizationVerifyListRequest . . . 9
3.12 struct AuthorizationVerifyListResponse . . . 9
3.13 struct AuthorizationVerifyRequest . . . 9
3.14 struct AuthorizationVerifyResponse . . . 10
3.15 struct MetadataRequirement . . . 10
3.16 struct Page . . . 10
3.17 Primitives . . . 10

4 References **12**

5 Revision History **13**
5.1 Amendments . . . 13
5.2 Quality Assurance . . . 13

1 Overview

This document describes the `authorizationManagement` service interface of the ConsumerAuthorization system over MQTT, which provides operator management of authorization policies at the management level, and bulk evaluation of authorization questions.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `authorizationManagement` service interface

This document provides the Interface Design Description IDD to the *authorizationManagement – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/consumer-authorization/authorization/management/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`grant-policies`	`arrowhead/consumer-authorization/authorization/management/grant-policies`
`revoke-policies`	`arrowhead/consumer-authorization/authorization/management/revoke-policies`
`query-policies`	`arrowhead/consumer-authorization/authorization/management/query-policies`
`check-policies`	`arrowhead/consumer-authorization/authorization/management/check-policies`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/consumer-authorization/authorization/management/grant-policies`

Operation: grant-policies
Input: AuthorizationMgmtGrantListRequest
Output: AuthorizationPolicyListResponse

Creates management-level policies for the given providers and targets.

```
{
  "list": [
    {
      "cloud": "LOCAL",
      "provider": "TemperatureSensor",
      "targetType": "SERVICE_DEF",
      "target": "temperatureReading",
```

```
      "description": "Display may read",
      "defaultPolicy": {
        "policyType": "WHITELIST",
        "policyList": [
          "TemperatureDisplay"
        ],
        "policyMetadataRequirement": {
          "unit": "celsius"
        }
      },
      "scopedPolicies": {
        "read": {
          "policyType": "ALL"
        }
      }
    }
  ]
}
```

Listing 3: Payload of an grant-policies request.

2.2 PUBLISH arrowhead/consumer-authorization/authorization/management/revoke-policies

Operation: revoke-policies
Input: Array<String>

Removes policies by instance identifier.

```
[
  "TemperatureSensor|temperatureReading|1.0.0"
]
```

Listing 4: Payload of an revoke-policies request.

2.3 PUBLISH arrowhead/consumer-authorization/authorization/management/query-policies

Operation: query-policies
Input: AuthorizationQueryRequest
Output: AuthorizationPolicyListResponse

Returns a page of policies of either level matching the filter.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "level": "MGMT",
  "providers": [
    "TemperatureSensor"
  ],
  "instanceIds": [
    "TemperatureSensor|temperatureReading|1.0.0"
  ],
  "cloudIdentifiers": [
    "string"
  ],
  "targetNames": [
    "string"
```

```
  ],
  "targetType": "SERVICE_DEF"
}
```

Listing 5: Payload of an query-policies request.

2.4 PUBLISH arrowhead/consumer-authorization/authorization/management/check-policies

Operation: check-policies
Input: AuthorizationVerifyListRequest
Output: AuthorizationVerifyListResponse

Evaluates a list of authorization questions and returns the answer for each.

```
{
  "list": [
    {
      "provider": "TemperatureSensor",
      "consumer": "TemperatureDisplay",
      "cloud": "LOCAL",
      "targetType": "SERVICE_DEF",
      "target": "temperatureReading",
      "scope": "read"
    }
  ]
}
```

Listing 6: Payload of an check-policies request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct AuthorizationMgmtGrantListRequest

Management-level policies to create.

Object Field	Value Type	Description
"list"	Array<AuthorizationMgmtGrantRequest>	The list of request items.

3.5 struct AuthorizationMgmtGrantRequest

A management-level authorization policy for one provider and target.

Object Field	Value Type	Description
"cloud"	String	Cloud identifier of the consumer cloud the policy applies to; `LOCAL` for the own cloud.
"provider"	String	Provider system the policy is set for.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"description"	String	Free-text description.
"defaultPolicy"	AuthorizationPolicyRequest	Policy that applies to all operations without a scoped policy.
"scopedPolicies"	Object<AuthorizationPolicyRequest>	Policies for individual operations, keyed by operation name.

3.6 struct AuthorizationPolicy

One policy rule.

Object Field	Value Type	Description
"policyType"	AuthorizationPolicyType	Policy type: `ALL`, `WHITELIST`, `BLACKLIST` or `SYS_METADATA`.
"policyList"	Array<String>	System names of the white- or blacklist.
"policyMetadata Requirement"	MetadataRequirement	Metadata the consumer system must have (`SYS_METADATA`).

3.7 struct AuthorizationPolicyListResponse

List of authorization policies.

Object Field	Value Type	Description
"entries"	Array<AuthorizationPolicy Response>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.8 struct AuthorizationPolicyRequest

One policy rule.

Object Field	Value Type	Description
"policyType"	String	Policy type: `ALL`, `WHITELIST`, `BLACKLIST` or `SYS_METADATA`.
"policyList"	Array<String>	System names of the white- or blacklist.
"policyMetadata Requirement"	MetadataRequirement	Metadata the consumer system must have (`SYS_METADATA`).

3.9 struct AuthorizationPolicyResponse

A stored authorization policy.

Object Field	Value Type	Description
"instanceId"	String	Policy instance identifier `<MGMT or PR>|<cloud>|<provider>|<targetType>|<target>`.
"level"	AuthorizationLevel	Policy level: `MGMT` (set by an operator) or `PROVIDER` (set by the provider itself).
"cloud"	String	Cloud identifier of the consumer cloud the policy applies to; `LOCAL` for the own cloud.
"provider"	String	Provider system the policy protects.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"description"	String	Free-text description.
"defaultPolicy"	AuthorizationPolicy	Policy that applies to all operations without a scoped policy.
"scopedPolicies"	Object<AuthorizationPolicy>	Policies for individual operations, keyed by operation name.
"createdBy"	String	Name of the system that created the record.
"createdAt"	DateTime	UTC time of creation.

3.10 struct AuthorizationQueryRequest

Filter for authorization policies.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"level"	String	Policy level to match, `MGMT` or `PROVIDER`.
"providers"	Array<String>	Provider system names to match.
"instanceIds"	Array<String>	Policy instance identifiers to match.
"cloudIdentifiers"	Array<String>	Cloud identifiers to match.
"targetNames"	Array<String>	Target names to match.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.

3.11 struct AuthorizationVerifyListRequest

Authorization checks to evaluate.

Object Field	Value Type	Description
"list"	Array<AuthorizationVerifyRequest>	The list of request items.

3.12 struct AuthorizationVerifyListResponse

Results of authorization checks.

Object Field	Value Type	Description
"entries"	Array<AuthorizationVerifyResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.13 struct AuthorizationVerifyRequest

One authorization question: may the consumer use the target (operation) of the provider?

Object Field	Value Type	Description
"provider"	String	Provider system; for `verify` defaults to the requester.
"consumer"	String	Consumer system; for `verify` defaults to the requester.
"cloud"	String	Cloud of the consumer, `LOCAL` by default.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"scope"	String	Operation to check; empty checks the default policy.

3.14 struct AuthorizationVerifyResponse

The answer to one authorization question.

Object Field	Value Type	Description
"provider"	String	Name of the provider system.
"consumer"	String	Name of the consumer system.
"cloud"	String	Cloud of the consumer.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"scope"	String	The checked operation.
"granted"	Boolean	Result of the check: true when the consumer is authorized.

3.15 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.16 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.17 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
AuthorizationLevel	One of `MGMT`, `PROVIDER` (JSON String).
AuthorizationPolicyType	One of `ALL`, `WHITELIST`, `BLACKLIST`, `SYS_METADATA` (JSON String).
AuthorizationTargetType	One of `SERVICE_DEF`, `EVENT_TYPE` (JSON String).
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “authorizationManagement – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorizationManagement-SD.pdf.

[6] ——, “authorizationManagement HTTP/TLS/JSON – Interface Design Description (ConsumerAuthorization),” Eclipse Arrowhead, IDD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorizationManagement-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

authorizationToken HTTP/TLS/JSON

Interface Design Description

Service ID: *"authorizationToken"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `authorizationToken` service of the ConsumerAuthorization system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation generate . . . 4
2.2 operation verify . . . 4
2.3 operation get-public-key . . . 5
2.4 operation register-encryption-key . . . 5
2.5 operation unregister-encryption-key . . . 6

3 Data Models **7**
3.1 struct AuthorizationEncryptionKeyRegistrationRequest . . . 7
3.2 struct AuthorizationTokenGenerationRequest . . . 7
3.3 struct AuthorizationTokenGenerationResponse . . . 7
3.4 struct AuthorizationTokenVerifyResponse . . . 7
3.5 struct ErrorMessage . . . 8
3.6 Primitives . . . 8

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `authorizationToken` service interface of the ConsumerAuthorization system, which provides authorization tokens: a consumer obtains a token for a specific provider, service and operation; the provider verifies it.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `authorizationToken` service interface

This document provides the Interface Design Description IDD to the *authorizationToken – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `authorizationToken` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8445 and base path `/consumerauthorization/authorization-token`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`generate`	POST	`/generate`
`verify`	GET	`/token/verify/{token}`
`get-public-key`	GET	`/public-key`
`register-encryption-key`	POST	`/encryption-key`
`unregister-encryption-key`	DELETE	`/encryption-key`

2 Service Operations

The interfaces of the `authorizationToken` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `authorizationToken` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /consumerauthorization/authorization-token/generate

Operation: generate
Input: AuthorizationTokenGenerationRequest
Output: AuthorizationTokenGenerationResponse

Generates a token of the requested variant for the requester (as consumer) to the provider's target and operation, if authorized.

A successful call returns `201 Created` with the output in the body.

```
POST /consumerauthorization/authorization-token/generate HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "tokenVariant": "TIME_LIMITED_TOKEN_AUTH",
  "provider": "TemperatureSensor",
  "targetType": "SERVICE_DEF",
  "target": "temperatureReading",
  "scope": "read"
}
```

Listing 1: An generate invocation.

```
{
  "tokenType": "TIME_LIMITED_TOKEN",
  "targetType": "SERVICE_DEF",
  "token": "<token>",
  "usageLimit": 5,
  "expiresAt": "2026-10-22T12:00:00Z"
}
```

Listing 2: An generate response.

2.2 GET /consumerauthorization/authorization-token/token/verify/*{token}*

Operation: verify
Input: String
Output: AuthorizationTokenVerifyResponse

Verifies a token presented to the requester (as provider), counts a use of usage-limited tokens and returns its consumer, target and scope.

Path parameter: *token*.
A successful call returns `200 OK` with the output in the body.

```
GET /consumerauthorization/authorization-token/token/verify/<token> HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An verify invocation.

```
{
  "verified": true,
  "consumerCloud": "LOCAL",
  "consumer": "TemperatureDisplay",
  "targetType": "SERVICE_DEF",
  "target": "temperatureReading",
  "scope": "read"
}
```

Listing 4: An verify response.

2.3 GET /consumerauthorization/authorization-token/public-key

Operation: get-public-key
Output: String

Returns the public key (Base64 X.509 SubjectPublicKeyInfo) with which providers verify JSON Web Tokens.
A successful call returns `200 OK` with the output in the body.

```
GET /consumerauthorization/authorization-token/public-key HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 5: An get-public-key invocation.

```
"string"
```

Listing 6: An get-public-key response.

2.4 POST /consumerauthorization/authorization-token/encryption-key

Operation: register-encryption-key
Input: AuthorizationEncryptionKeyRegistrationRequest
Output: String

Registers or replaces the requester's AES key for encrypting its self-contained tokens. Returns the initialization vector (CBC) or an empty string; 201 for a new key.
A successful call returns `201 Created` with the output in the body.

```
POST /consumerauthorization/authorization-token/encryption-key HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "key": "0123456789abcdef",
  "algorithm": "AES/ECB/PKCS5Padding"
}
```

Listing 7: An register-encryption-key invocation.

```
"string"
```

Listing 8: An register-encryption-key response.

2.5 DELETE /consumerauthorization/authorization-token/encryption-key

Operation: unregister-encryption-key

Removes the requester's encryption key. Returns 200 if removed, 204 if there was none.

A successful call returns `200 OK` if the entry was removed, `204 No Content` if there was nothing to remove.

```
DELETE /consumerauthorization/authorization-token/encryption-key HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 9: An unregister-encryption-key invocation.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.6, which are used to represent things like hashes, identifiers and texts.

3.1 struct AuthorizationEncryptionKeyRegistrationRequest

Registers the requester's key for encrypting its self-contained tokens.

Object Field	Value Type	Description
"key"	String	AES key of the provider (at least 16 bytes).
"algorithm"	String	Encryption algorithm of the key, e.g. `AES/ECB/PKCS5Padding` or `AES/CBC/PKCS5Padding`.

3.2 struct AuthorizationTokenGenerationRequest

Request of the consumer for a token to one provider's service operation.

Object Field	Value Type	Description
"tokenVariant"	String	Requested token variant, as interface security policy (e.g. `TIME_LIMITED_TOKEN_AUTH`, `RSA_SHA512_JSON_WEB_TOKEN_AUTH`).
"provider"	String	Provider system the token is for.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Service definition (or event type).
"scope"	String	Operation the token is for.

3.3 struct AuthorizationTokenGenerationResponse

A generated token.

Object Field	Value Type	Description
"tokenType"	AuthorizationTokenType	Token type: `TIME_LIMITED_TOKEN`, `USAGE_LIMITED_TOKEN`, `SELF_CONTAINED_TOKEN` or `TRANSLATION_BRIDGE_TOKEN`.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"token"	String	The token (opaque string, Base64 or JSON Web Token).
"usageLimit"	Number	Number of allowed uses (usage-limited tokens).
"expiresAt"	DateTime	UTC expiry of time-limited tokens.

3.4 struct AuthorizationTokenVerifyResponse

Result of verifying a token presented to a provider.

Object Field	Value Type	Description
"verified"	Boolean	True when the token is valid.
"consumerCloud"	String	Cloud identifier of the consumer system.
"consumer"	String	Name of the consumer system.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"scope"	String	Operation the token or policy is limited to; empty for all operations.

3.5 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.6 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
AuthorizationTargetType	One of `SERVICE_DEF`, `EVENT_TYPE` (JSON String).
AuthorizationTokenType	One of `TIME_LIMITED_TOKEN`, `USAGE_LIMITED_TOKEN`, `SELF_CONTAINED_TOKEN`, `TRANSLATION_BRIDGE_TOKEN` (JSON String).
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “authorizationToken – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorizationToken-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

authorizationToken MQTT/TLS/JSON

Interface Design Description

Service ID: *"authorizationToken"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `authorizationToken` service of the ConsumerAuthorization system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation generate . . . 4
2.2 operation verify . . . 5
2.3 operation get-public-key . . . 5
2.4 operation register-encryption-key . . . 5
2.5 operation unregister-encryption-key . . . 5

3 Data Models **6**
3.1 struct MqttRequestTemplate . . . 6
3.2 struct MqttResponseTemplate . . . 6
3.3 struct ErrorMessage . . . 6
3.4 struct AuthorizationEncryptionKeyRegistrationRequest . . . 6
3.5 struct AuthorizationTokenGenerationRequest . . . 6
3.6 struct AuthorizationTokenGenerationResponse . . . 7
3.7 struct AuthorizationTokenVerifyResponse . . . 7
3.8 Primitives . . . 7

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `authorizationToken` service interface of the ConsumerAuthorization system over MQTT, which provides authorization tokens: a consumer obtains a token for a specific provider, service and operation; the provider verifies it.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `authorizationToken` service interface

This document provides the Interface Design Description IDD to the *authorizationToken – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/consumer-authorization/authorization-token/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`generate`	`arrowhead/consumer-authorization/authorization-token/generate`
`verify`	`arrowhead/consumer-authorization/authorization-token/verify`
`get-public-key`	`arrowhead/consumer-authorization/authorization-token/get-public-key`
`register-encryption-key`	`arrowhead/consumer-authorization/authorization-token/register-encryption-key`
`unregister-encryption-key`	`arrowhead/consumer-authorization/authorization-token/unregister-encryption-key`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/consumer-authorization/authorization-token/generate`

Operation: generate
Input: AuthorizationTokenGenerationRequest
Output: AuthorizationTokenGenerationResponse

Generates a token of the requested variant for the requester (as consumer) to the provider's target and operation, if authorized.

```
{
  "tokenVariant": "TIME_LIMITED_TOKEN_AUTH",
  "provider": "TemperatureSensor",
```

```
  "targetType": "SERVICE_DEF",
  "target": "temperatureReading",
  "scope": "read"
}
```

Listing 3: Payload of an generate request.

2.2 PUBLISH arrowhead/consumer-authorization/authorization-token/verify

Operation: verify
Input: String
Output: AuthorizationTokenVerifyResponse

Verifies a token presented to the requester (as provider), counts a use of usage-limited tokens and returns its consumer, target and scope.

```
"<token>"
```

Listing 4: Payload of an verify request.

2.3 PUBLISH arrowhead/consumer-authorization/authorization-token/get-public-key

Operation: get-public-key
Output: String

Returns the public key (Base64 X.509 SubjectPublicKeyInfo) with which providers verify JSON Web Tokens.

2.4 PUBLISH arrowhead/consumer-authorization/authorization-token/register-encryption-key

Operation: register-encryption-key
Input: AuthorizationEncryptionKeyRegistrationRequest
Output: String

Registers or replaces the requester's AES key for encrypting its self-contained tokens. Returns the initialization vector (CBC) or an empty string; 201 for a new key.

```
{
  "key": "0123456789abcdef",
  "algorithm": "AES/ECB/PKCS5Padding"
}
```

Listing 5: Payload of an register-encryption-key request.

2.5 PUBLISH arrowhead/consumer-authorization/authorization-token/unregister-encryption-key

Operation: unregister-encryption-key

Removes the requester's encryption key. Returns 200 if removed, 204 if there was none.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct AuthorizationEncryptionKeyRegistrationRequest

Registers the requester's key for encrypting its self-contained tokens.

Object Field	Value Type	Description
"key"	String	AES key of the provider (at least 16 bytes).
"algorithm"	String	Encryption algorithm of the key, e.g. `AES/ECB/PKCS5Padding` or `AES/CBC/PKCS5Padding`.

3.5 struct AuthorizationTokenGenerationRequest

Request of the consumer for a token to one provider's service operation.

Object Field	Value Type	Description
"tokenVariant"	String	Requested token variant, as interface security policy (e.g. `TIME_LIMITED_TOKEN_AUTH`, `RSA_SHA512_JSON_WEB_TOKEN_AUTH`).
"provider"	String	Provider system the token is for.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Service definition (or event type).
"scope"	String	Operation the token is for.

3.6 struct AuthorizationTokenGenerationResponse

A generated token.

Object Field	Value Type	Description
"tokenType"	AuthorizationTokenType	Token type: `TIME_LIMITED_TOKEN`, `USAGE_LIMITED_TOKEN`, `SELF_CONTAINED_TOKEN` or `TRANSLATION_BRIDGE_TOKEN`.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"token"	String	The token (opaque string, Base64 or JSON Web Token).
"usageLimit"	Number	Number of allowed uses (usage-limited tokens).
"expiresAt"	DateTime	UTC expiry of time-limited tokens.

3.7 struct AuthorizationTokenVerifyResponse

Result of verifying a token presented to a provider.

Object Field	Value Type	Description
"verified"	Boolean	True when the token is valid.
"consumerCloud"	String	Cloud identifier of the consumer system.
"consumer"	String	Name of the consumer system.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"scope"	String	Operation the token or policy is limited to; empty for all operations.

3.8 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
AuthorizationTargetType	One of `SERVICE_DEF`, `EVENT_TYPE` (JSON String).
AuthorizationTokenType	One of `TIME_LIMITED_TOKEN`, `USAGE_LIMITED_TOKEN`, `SELF_CONTAINED_TOKEN`, `TRANSLATION_BRIDGE_TOKEN` (JSON String).
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “authorizationToken – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorizationToken-SD.pdf.

[6] ——, “authorizationToken HTTP/TLS/JSON – Interface Design Description (ConsumerAuthorization),” Eclipse Arrowhead, IDD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorizationToken-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

authorizationTokenManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"authorizationTokenManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `authorizationTokenManagement` service of the ConsumerAuthorization system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation generate-tokens . . . 4
2.2 operation query-tokens . . . 5
2.3 operation revoke-tokens . . . 6
2.4 operation add-encryption-keys . . . 6
2.5 operation remove-encryption-keys . . . 7

3 Data Models **8**
3.1 struct AuthorizationMgmtEncryptionKeyListResponse . . . 8
3.2 struct AuthorizationMgmtEncryptionKeyRegistrationListRequest . . . 8
3.3 struct AuthorizationMgmtEncryptionKeyRegistrationRequest . . . 8
3.4 struct AuthorizationMgmtEncryptionKeyResponse . . . 8
3.5 struct AuthorizationTokenGenerationMgmtListRequest . . . 8
3.6 struct AuthorizationTokenGenerationMgmtRequest . . . 9
3.7 struct AuthorizationTokenMgmtListResponse . . . 9
3.8 struct AuthorizationTokenQueryRequest . . . 9
3.9 struct AuthorizationTokenResponse . . . 10
3.10 struct ErrorMessage . . . 10
3.11 struct Page . . . 10
3.12 Primitives . . . 11

4 References **12**

5 Revision History **13**
5.1 Amendments . . . 13
5.2 Quality Assurance . . . 13

1 Overview

This document describes the `authorizationTokenManagement` service interface of the ConsumerAuthorization system, which provides operator management of authorization tokens and of provider encryption keys.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `authorizationTokenManagement` service interface

This document provides the Interface Design Description IDD to the *authorizationTokenManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `authorizationTokenManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8445 and base path `/consumerauthorization/authorization/mgmt/token`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`generate-tokens`	POST	`/generate`
`query-tokens`	POST	`/query`
`revoke-tokens`	DELETE	`/revoke`
`add-encryption-keys`	POST	`/encryption-key`
`remove-encryption-keys`	DELETE	`/encryption-key`

2 Service Operations

The interfaces of the `authorizationTokenManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `authorizationTokenManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /consumerauthorization/authorization/mgmt/token/generate?unbound=*{unbound}*

Operation: generate-tokens
Input: AuthorizationTokenGenerationMgmtListRequest
Output: AuthorizationTokenMgmtListResponse

Generates tokens on behalf of consumers. With `unbound=true` the authorization check is skipped (only for systems in `unbounded.token.generation.whitelist`).

Query parameters: *unbound* (boolean, optional, default `false`). List values are repeated or comma-separated.

A successful call returns `201 Created` with the output in the body.

```
POST /consumerauthorization/authorization/mgmt/token/generate?unbound=<unbound> HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "list": [
    {
      "tokenVariant": "TIME_LIMITED_TOKEN_AUTH",
      "targetType": "SERVICE_DEF",
      "consumerCloud": "LOCAL",
      "consumer": "TemperatureDisplay",
      "provider": "TemperatureSensor",
      "target": "temperatureReading",
      "scope": "read",
      "expiresAt": "2026-10-22T12:00:00Z",
      "usageLimit": 5
    }
  ]
}
```

Listing 1: An generate-tokens invocation.

```
{
  "entries": [
    {
      "tokenType": "TIME_LIMITED_TOKEN",
      "variant": "TIME_LIMITED_TOKEN_AUTH",
      "token": "<token>",
      "tokenReference": "<token reference>",
      "requester": "DynamicServiceOrchestration",
```

```
      "consumerCloud": "LOCAL",
      "consumer": "TemperatureDisplay",
      "provider": "TemperatureSensor",
      "targetType": "SERVICE_DEF",
      "target": "temperatureReading",
      "scope": "read",
      "createdAt": "2026-10-22T12:00:00Z",
      "usageLimit": 5,
      "usageLeft": 5,
      "expiresAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 2: An generate-tokens response.

2.2 POST /consumerauthorization/authorization/mgmt/token/query

Operation: query-tokens
Input: AuthorizationTokenQueryRequest
Output: AuthorizationTokenMgmtListResponse

Returns a page of tokens matching the filter.

A successful call returns `200 OK` with the output in the body.

```
POST /consumerauthorization/authorization/mgmt/token/query HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "requester": "DynamicServiceOrchestration",
  "tokenType": "TIME_LIMITED_TOKEN",
  "consumerCloud": "LOCAL",
  "consumer": "TemperatureDisplay",
  "provider": "TemperatureSensor",
  "targetType": "SERVICE_DEF",
  "target": "temperatureReading"
}
```

Listing 3: An query-tokens invocation.

```
{
  "entries": [
    {
      "tokenType": "TIME_LIMITED_TOKEN",
      "variant": "TIME_LIMITED_TOKEN_AUTH",
      "token": "<token>",
      "tokenReference": "<token reference>",
      "requester": "DynamicServiceOrchestration",
      "consumerCloud": "LOCAL",
      "consumer": "TemperatureDisplay",
      "provider": "TemperatureSensor",
      "targetType": "SERVICE_DEF",
      "target": "temperatureReading",
      "scope": "read",
      "createdAt": "2026-10-22T12:00:00Z",
```

```
      "usageLimit": 5,
      "usageLeft": 5,
      "expiresAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 4: An query-tokens response.

2.3 DELETE /consumerauthorization/authorization/mgmt/token/revoke?tokenReferences=*{tokenReferences}*

Operation: revoke-tokens
Input: Array<String>

Revokes tokens by token reference.

Query parameters: *tokenReferences* (List<String>, required). List values are repeated or comma-separated.

A successful call returns `200 OK`.

```
DELETE /consumerauthorization/authorization/mgmt/token/revoke?tokenReferences=<tokenReferences> HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 5: An revoke-tokens invocation.

2.4 POST /consumerauthorization/authorization/mgmt/token/encryption-key

Operation: add-encryption-keys
Input: AuthorizationMgmtEncryptionKeyRegistrationListRequest
Output: AuthorizationMgmtEncryptionKeyListResponse

Registers encryption keys on behalf of providers.

A successful call returns `201 Created` with the output in the body.

```
POST /consumerauthorization/authorization/mgmt/token/encryption-key HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "list": [
    {
      "systemName": "TemperatureSensor",
      "key": "0123456789abcdef",
      "algorithm": "AES/ECB/PKCS5Padding"
    }
  ]
}
```

Listing 6: An add-encryption-keys invocation.

```
{
  "entries": [
    {
      "systemName": "TemperatureSensor",
      "rawKey": "0123456789abcdef",
      "algorithm": "AES/ECB/PKCS5Padding",
      "keyAdditive": "",
```

```
      "createdAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 7: An add-encryption-keys response.

2.5 DELETE /consumerauthorization/authorization/mgmt/token/encryption-key?systemNames=*{systemNames}*

Operation: remove-encryption-keys
Input: Array<String>

Removes the encryption keys of the named providers.
Query parameters: *systemNames* (List<String>, required). List values are repeated or comma-separated.
A successful call returns `200 OK`.

```
DELETE /consumerauthorization/authorization/mgmt/token/encryption-key?systemNames=<systemNames> HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 8: An remove-encryption-keys invocation.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.12, which are used to represent things like hashes, identifiers and texts.

3.1 struct AuthorizationMgmtEncryptionKeyListResponse

List of registered encryption keys.

Object Field	Value Type	Description
"entries"	Array<AuthorizationMgmtEncryptionKeyResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.2 struct AuthorizationMgmtEncryptionKeyRegistrationListRequest

Encryption keys to register on behalf of providers.

Object Field	Value Type	Description
"list"	Array<AuthorizationMgmtEncryptionKeyRegistrationRequest>	The list of request items.

3.3 struct AuthorizationMgmtEncryptionKeyRegistrationRequest

Encryption key of one provider.

Object Field	Value Type	Description
"systemName"	String	Provider system the key belongs to.
"key"	String	The encryption key, as a string.
"algorithm"	String	Encryption algorithm of the key, e.g. `AES/ECB/PKCS5Padding` or `AES/CBC/PKCS5Padding`.

3.4 struct AuthorizationMgmtEncryptionKeyResponse

A registered encryption key.

Object Field	Value Type	Description
"systemName"	String	Name of the system, PascalCase (e.g. `TemperatureSensor`).
"rawKey"	String	The registered encryption key.
"algorithm"	String	Encryption algorithm of the key, e.g. `AES/ECB/PKCS5Padding` or `AES/CBC/PKCS5Padding`.
"keyAdditive"	String	Initialization vector (CBC mode), Base64; empty for ECB.
"createdAt"	DateTime	UTC time of creation.

3.5 struct AuthorizationTokenGenerationMgmtListRequest

Tokens to generate on behalf of consumers.

Object Field	Value Type	Description
"list"	Array<AuthorizationTokenGenerationMgmtRequest>	The list of request items.

3.6 struct AuthorizationTokenGenerationMgmtRequest

One token to generate on behalf of a consumer.

Object Field	Value Type	Description
"tokenVariant"	String	Requested token variant, as interface security policy (e.g. `TIME_LIMITED_TOKEN_AUTH`, `RSA_SHA512_JSON_WEB_TOKEN_AUTH`).
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"consumerCloud"	String	Cloud identifier of the consumer system.
"consumer"	String	Name of the consumer system.
"provider"	String	Name of the provider system.
"target"	String	Target of the policy or token: a service definition or event type.
"scope"	String	Operation the token or policy is limited to; empty for all operations.
"expiresAt"	DateTime	Requested expiry; default `token.time.limit` from now.
"usageLimit"	Number	Requested uses; default `simple.token.usage.limit`.

3.7 struct AuthorizationTokenMgmtListResponse

List of generated tokens.

Object Field	Value Type	Description
"entries"	Array<AuthorizationTokenResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.8 struct AuthorizationTokenQueryRequest

Filter for generated tokens.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"requester"	String	Name of the system that requested the token.
"tokenType"	String	Token type to match.
"consumerCloud"	String	Cloud identifier of the consumer system.
"consumer"	String	Name of the consumer system.
"provider"	String	Name of the provider system.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.

3.9 struct AuthorizationTokenResponse

A generated token with its bookkeeping data.

Object Field	Value Type	Description
"tokenType"	AuthorizationTokenType	Token type: `TIME_LIMITED_TOKEN`, `USAGE_LIMITED_TOKEN`, `SELF_CONTAINED_TOKEN` or `TRANSLATION_BRIDGE_TOKEN`.
"variant"	String	Token variant (interface security policy).
"token"	String	The token, for simple tokens only in hashed form.
"tokenReference"	String	Reference of the token, used to revoke it.
"requester"	String	Name of the system that requested the token.
"consumerCloud"	String	Cloud identifier of the consumer system.
"consumer"	String	Name of the consumer system.
"provider"	String	Name of the provider system.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"scope"	String	Operation the token or policy is limited to; empty for all operations.
"createdAt"	DateTime	UTC time of creation.
"usageLimit"	Number	Number of allowed uses (usage-limited tokens).
"usageLeft"	Number	Remaining uses of a usage-limited token.
"expiresAt"	DateTime	UTC expiry time; no value means no expiry.

3.10 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.11 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.12 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
AuthorizationTargetType	One of `SERVICE_DEF`, `EVENT_TYPE` (JSON String).
AuthorizationTokenType	One of `TIME_LIMITED_TOKEN`, `USAGE_LIMITED_TOKEN`, `SELF_CONTAINED_TOKEN`, `TRANSLATION_BRIDGE_TOKEN` (JSON String).
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “authorizationTokenManagement – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorizationTokenManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

authorizationTokenManagement MQTT/TLS/JSON

Interface Design Description

Service ID: *"authorizationTokenManagement"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `authorizationTokenManagement` service of the ConsumerAuthorization system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation generate-tokens . . . 4
2.2 operation query-tokens . . . 5
2.3 operation revoke-tokens . . . 5
2.4 operation add-encryption-keys . . . 5
2.5 operation remove-encryption-keys . . . 6

3 Data Models **7**
3.1 struct MqttRequestTemplate . . . 7
3.2 struct MqttResponseTemplate . . . 7
3.3 struct ErrorMessage . . . 7
3.4 struct AuthorizationMgmtEncryptionKeyListResponse . . . 7
3.5 struct AuthorizationMgmtEncryptionKeyRegistrationListRequest . . . 7
3.6 struct AuthorizationMgmtEncryptionKeyRegistrationRequest . . . 8
3.7 struct AuthorizationMgmtEncryptionKeyResponse . . . 8
3.8 struct AuthorizationTokenGenerationMgmtListRequest . . . 8
3.9 struct AuthorizationTokenGenerationMgmtRequest . . . 8
3.10 struct AuthorizationTokenMgmtListResponse . . . 9
3.11 struct AuthorizationTokenQueryRequest . . . 9
3.12 struct AuthorizationTokenResponse . . . 9
3.13 struct Page . . . 10
3.14 Primitives . . . 10

4 References **12**

5 Revision History **13**
5.1 Amendments . . . 13
5.2 Quality Assurance . . . 13

1 Overview

This document describes the `authorizationTokenManagement` service interface of the ConsumerAuthorization system over MQTT, which provides operator management of authorization tokens and of provider encryption keys.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `authorizationTokenManagement` service interface

This document provides the Interface Design Description IDD to the *authorizationTokenManagement – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/consumerauthorization/authorization-token/management/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`generate-tokens`	`arrowhead/consumer-authorization/authorization-token/management/generate-tokens`
`query-tokens`	`arrowhead/consumer-authorization/authorization-token/management/query-tokens`
`revoke-tokens`	`arrowhead/consumer-authorization/authorization-token/management/revoke-tokens`
`add-encryption-keys`	`arrowhead/consumer-authorization/authorization-token/management/add-encryption-keys`
`remove-encryption-keys`	`arrowhead/consumer-authorization/authorization-token/management/remove-encryption-keys`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/consumer-authorization/authorization-token/management/generate-tokens`

Operation: generate-tokens
Input: AuthorizationTokenGenerationMgmtListRequest
Output: AuthorizationTokenMgmtListResponse

Generates tokens on behalf of consumers. With `unbound=true` the authorization check is skipped (only for systems in `unbounded.token.generation.whitelist`).

Optional entries of `params`: `unbound`.

```
{
  "list": [
```

```
    {
      "tokenVariant": "TIME_LIMITED_TOKEN_AUTH",
      "targetType": "SERVICE_DEF",
      "consumerCloud": "LOCAL",
      "consumer": "TemperatureDisplay",
      "provider": "TemperatureSensor",
      "target": "temperatureReading",
      "scope": "read",
      "expiresAt": "2026-10-22T12:00:00Z",
      "usageLimit": 5
    }
  ]
}
```

Listing 3: Payload of an generate-tokens request.

2.2 PUBLISH arrowhead/consumer-authorization/authorization-token/management/query-tokens

Operation: query-tokens
Input: AuthorizationTokenQueryRequest
Output: AuthorizationTokenMgmtListResponse

Returns a page of tokens matching the filter.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "requester": "DynamicServiceOrchestration",
  "tokenType": "TIME_LIMITED_TOKEN",
  "consumerCloud": "LOCAL",
  "consumer": "TemperatureDisplay",
  "provider": "TemperatureSensor",
  "targetType": "SERVICE_DEF",
  "target": "temperatureReading"
}
```

Listing 4: Payload of an query-tokens request.

2.3 PUBLISH arrowhead/consumer-authorization/authorization-token/management/revoke-tokens

Operation: revoke-tokens
Input: Array<String>

Revokes tokens by token reference.

```
[
  "<token reference>"
]
```

Listing 5: Payload of an revoke-tokens request.

2.4 PUBLISH arrowhead/consumer-authorization/authorization-token/management/add-encryption-keys

Operation: add-encryption-keys
Input: AuthorizationMgmtEncryptionKeyRegistrationListRequest
Output: AuthorizationMgmtEncryptionKeyListResponse

Registers encryption keys on behalf of providers.

```
{
  "list": [
    {
      "systemName": "TemperatureSensor",
      "key": "0123456789abcdef",
      "algorithm": "AES/ECB/PKCS5Padding"
    }
  ]
}
```

Listing 6: Payload of an add-encryption-keys request.

2.5 PUBLISH arrowhead/consumer-authorization/authorization-token/management/remove-encryption-keys

Operation: remove-encryption-keys
Input: Array<String>

Removes the encryption keys of the named providers.

```
[
  "TemperatureSensor"
]
```

Listing 7: Payload of an remove-encryption-keys request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct AuthorizationMgmtEncryptionKeyListResponse

List of registered encryption keys.

Object Field	Value Type	Description
"entries"	Array<AuthorizationMgmtEncryptionKeyResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.5 struct AuthorizationMgmtEncryptionKeyRegistrationListRequest

Encryption keys to register on behalf of providers.

Object Field	Value Type	Description
"list"	Array<AuthorizationMgmtEncryption KeyRegistrationRequest>	The list of request items.

3.6 struct AuthorizationMgmtEncryptionKeyRegistrationRequest

Encryption key of one provider.

Object Field	Value Type	Description
"systemName"	String	Provider system the key belongs to.
"key"	String	The encryption key, as a string.
"algorithm"	String	Encryption algorithm of the key, e.g. `AES/ECB/PKCS5Padding` or `AES/CBC/PKCS5Padding`.

3.7 struct AuthorizationMgmtEncryptionKeyResponse

A registered encryption key.

Object Field	Value Type	Description
"systemName"	String	Name of the system, PascalCase (e.g. `TemperatureSensor`).
"rawKey"	String	The registered encryption key.
"algorithm"	String	Encryption algorithm of the key, e.g. `AES/ECB/PKCS5Padding` or `AES/CBC/PKCS5Padding`.
"keyAdditive"	String	Initialization vector (CBC mode), Base64; empty for ECB.
"createdAt"	DateTime	UTC time of creation.

3.8 struct AuthorizationTokenGenerationMgmtListRequest

Tokens to generate on behalf of consumers.

Object Field	Value Type	Description
"list"	Array<AuthorizationTokenGeneration MgmtRequest>	The list of request items.

3.9 struct AuthorizationTokenGenerationMgmtRequest

One token to generate on behalf of a consumer.

Object Field	Value Type	Description
"tokenVariant"	String	Requested token variant, as interface security policy (e.g. `TIME_LIMITED_TOKEN_AUTH`, `RSA_SHA512_JSON_WEB_TOKEN_AUTH`).
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"consumerCloud"	String	Cloud identifier of the consumer system.
"consumer"	String	Name of the consumer system.
"provider"	String	Name of the provider system.
"target"	String	Target of the policy or token: a service definition or event type.
"scope"	String	Operation the token or policy is limited to; empty for all operations.
"expiresAt"	DateTime	Requested expiry; default `token.time.limit` from now.
"usageLimit"	Number	Requested uses; default `simple.token.usage.limit`.

3.10 struct AuthorizationTokenMgmtListResponse

List of generated tokens.

Object Field	Value Type	Description
"entries"	Array<AuthorizationTokenResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.11 struct AuthorizationTokenQueryRequest

Filter for generated tokens.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"requester"	String	Name of the system that requested the token.
"tokenType"	String	Token type to match.
"consumerCloud"	String	Cloud identifier of the consumer system.
"consumer"	String	Name of the consumer system.
"provider"	String	Name of the provider system.
"targetType"	String	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.

3.12 struct AuthorizationTokenResponse

A generated token with its bookkeeping data.

Object Field	Value Type	Description
"tokenType"	AuthorizationTokenType	Token type: `TIME_LIMITED_TOKEN`, `USAGE_LIMITED_TOKEN`, `SELF_CONTAINED_TOKEN` or `TRANSLATION_BRIDGE_TOKEN`.
"variant"	String	Token variant (interface security policy).
"token"	String	The token, for simple tokens only in hashed form.
"tokenReference"	String	Reference of the token, used to revoke it.
"requester"	String	Name of the system that requested the token.
"consumerCloud"	String	Cloud identifier of the consumer system.
"consumer"	String	Name of the consumer system.
"provider"	String	Name of the provider system.
"targetType"	AuthorizationTargetType	Target type: `SERVICE_DEF` or `EVENT_TYPE`.
"target"	String	Target of the policy or token: a service definition or event type.
"scope"	String	Operation the token or policy is limited to; empty for all operations.
"createdAt"	DateTime	UTC time of creation.
"usageLimit"	Number	Number of allowed uses (usage-limited tokens).
"usageLeft"	Number	Remaining uses of a usage-limited token.
"expiresAt"	DateTime	UTC expiry time; no value means no expiry.

3.13 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.14 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
AuthorizationTargetType	One of `SERVICE_DEF`, `EVENT_TYPE` (JSON String).
AuthorizationTokenType	One of `TIME_LIMITED_TOKEN`, `USAGE_LIMITED_TOKEN`, `SELF_CONTAINED_TOKEN`, `TRANSLATION_BRIDGE_TOKEN` (JSON String).
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “authorizationTokenManagement – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorizationTokenManagement-SD.pdf.

[6] ——, “authorizationTokenManagement HTTP/TLS/JSON – Interface Design Description (ConsumerAuthorization),” Eclipse Arrowhead, IDD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorizationTokenManagement-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

generalManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"generalManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `generalManagement` service of the ConsumerAuthorization system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation get-log . . . 4
2.2 operation get-config . . . 4

3 Data Models **6**
3.1 struct ErrorMessage . . . 6
3.2 struct KeyValues . . . 6
3.3 struct LogEntry . . . 6
3.4 struct LogEntryListResponse . . . 6
3.5 struct LogRequest . . . 6
3.6 struct Page . . . 7
3.7 Primitives . . . 7

4 References **8**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the `generalManagement` service interface of the ConsumerAuthorization system, which provides general management of the ConsumerAuthorization system: reading its log entries and its configuration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `generalManagement` service interface

This document provides the Interface Design Description IDD to the *generalManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `generalManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8445 and base path `/consumerauthorization/general/mgmt`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`get-log`	POST	`/logs`
`get-config`	GET	`/get-config`

2 Service Operations

The interfaces of the `generalManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `generalManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /consumerauthorization/general/mgmt/logs

Operation: get-log
Input: LogRequest
Output: LogEntryListResponse

Returns a page of the system's log entries matching the filter.

A successful call returns `200 OK` with the output in the body.

```
POST /consumerauthorization/general/mgmt/logs HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "from": "2026-10-22T12:00:00Z",
  "to": "2026-10-22T12:00:00Z",
  "severity": "INFO",
  "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService"
}
```

Listing 1: An get-log invocation.

```
{
  "entries": [
    {
      "logId": "42",
      "entryDate": "2026-10-22T12:00:00Z",
      "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService",
      "severity": "INFO",
      "message": "Service instance registered: TemperatureSensor|temperatureReading|1.0.0"
    }
  ],
  "count": 1
}
```

Listing 2: An get-log response.

2.2 GET /consumerauthorization/general/mgmt/get-config?keys=*{keys}*

Operation: get-config
Output: KeyValues

Returns the effective values of the requested configuration keys. Only the public keys of the system are answered; other keys, including all secrets, are ignored.

Query parameters: *keys* (List<String>, optional). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
GET /consumerauthorization/general/mgmt/get-config?keys=<keys> HTTP/1.1
Host: consumerauthorization:8445
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An get-config invocation.

```
{
  "map": {
    "server.port": "8443"
  }
}
```

Listing 4: An get-config response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.7, which are used to represent things like hashes, identifiers and texts.

3.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.2 struct KeyValues

Configuration values of the system.

Object Field	Value Type	Description
"map"	Object<String>	The configuration keys and values; secrets are not returned.

3.3 struct LogEntry

One log entry.

Object Field	Value Type	Description
"logId"	String	Identifier of the log entry.
"entryDate"	DateTime	UTC time of the log entry.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.
"severity"	String	Log level, e.g. `INFO`, `WARN`, `ERROR`; as filter, the minimum level.
"message"	String	Text of the log entry or status message of the job.
"exception"	String	Stack trace or exception text of the log entry, if any.

3.4 struct LogEntryListResponse

List of log entries.

Object Field	Value Type	Description
"entries"	Array<LogEntry>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.5 struct LogRequest

Paged filter for the log entries stored in the system's database.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"from"	DateTime	Return entries from this UTC time.
"to"	DateTime	Return entries until this UTC time.
"severity"	String	Minimum log level to return.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.

3.6 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.7 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “generalManagement – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-generalManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

generalManagement MQTT/TLS/JSON

Interface Design Description

Service ID: *"generalManagement"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `generalManagement` service of the ConsumerAuthorization system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation get-log . . . 4
2.2 operation get-config . . . 5

3 Data Models **6**
3.1 struct MqttRequestTemplate . . . 6
3.2 struct MqttResponseTemplate . . . 6
3.3 struct ErrorMessage . . . 6
3.4 struct KeyValues . . . 6
3.5 struct LogEntry . . . 6
3.6 struct LogEntryListResponse . . . 7
3.7 struct LogRequest . . . 7
3.8 struct Page . . . 7
3.9 Primitives . . . 7

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `generalManagement` service interface of the ConsumerAuthorization system over MQTT, which provides general management of the ConsumerAuthorization system: reading its log entries and its configuration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `generalManagement` service interface

This document provides the Interface Design Description IDD to the *generalManagement – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/consumerauthorization/general/management/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`get-log`	`arrowhead/consumer-authorization/general/management/get-log`
`get-config`	`arrowhead/consumer-authorization/general/management/get-config`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/consumer-authorization/general/management/get-log`

Operation: get-log
Input: LogRequest
Output: LogEntryListResponse

Returns a page of the system's log entries matching the filter.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "from": "2026-10-22T12:00:00Z",
  "to": "2026-10-22T12:00:00Z",
  "severity": "INFO",
  "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService"
}
```

Listing 3: Payload of an get-log request.

2.2 PUBLISH `arrowhead/consumer-authorization/general/management/get-config`

Operation: get-config
Input: Array<String>
Output: KeyValues

Returns the effective values of the requested configuration keys. Only the public keys of the system are answered; other keys, including all secrets, are ignored.

```
[
  "string"
]
```

Listing 4: Payload of an get-config request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct KeyValues

Configuration values of the system.

Object Field	Value Type	Description
"map"	Object<String>	The configuration keys and values; secrets are not returned.

3.5 struct LogEntry

One log entry.

Object Field	Value Type	Description
"logId"	String	Identifier of the log entry.
"entryDate"	DateTime	UTC time of the log entry.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.
"severity"	String	Log level, e.g. `INFO`, `WARN`, `ERROR`; as filter, the minimum level.
"message"	String	Text of the log entry or status message of the job.
"exception"	String	Stack trace or exception text of the log entry, if any.

3.6 struct LogEntryListResponse

List of log entries.

Object Field	Value Type	Description
"entries"	Array<LogEntry>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.7 struct LogRequest

Paged filter for the log entries stored in the system's database.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"from"	DateTime	Return entries from this UTC time.
"to"	DateTime	Return entries until this UTC time.
"severity"	String	Minimum log level to return.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.

3.8 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.9 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “generalManagement – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-generalManagement-SD.pdf.

[6] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (ConsumerAuthorization),” Eclipse Arrowhead, IDD 5.2.2, 2026, consumerAuthorization-v5.2.2-generalManagement-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

sysMonitor AMQP/TLS/JSON

Interface Design Description

Service ID: *"sysMonitor"*

Abstract
This document describes the AMQP 0-9-1 protocol with TLS payload security and JSON payload encoding variant of the `sysMonitor` service of the ConsumerAuthorization system (SysMonitor-AMQP-IDD-5.2.2).

Contents

1 Overview **3**
1.1 Interface registration 3

2 Service Operations **4**
2.1 operation echo 4
2.2 operation power 4
2.3 operation status 4

3 Data Models **6**
3.1 struct Observation 6
3.2 Primitives 6

4 References **7**

5 Revision History **8**
5.1 Amendments 8
5.2 Quality Assurance 8

1 Overview

This document describes the AMQP binding of the native `sysMonitor` service of ConsumerAuthorization.
It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	AMQP	0-9-1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `sysMonitor` service interface

This document provides the Interface Design Description IDD to the *sysMonitor – Service Description* document [5].

1.1 Interface registration

With `sysmonitor.enabled=true`, ConsumerAuthorization registers the service at start-up with interface template `generic_amqps` (protocol `amqps`, policy `CERT_AUTH`; `generic_amqp` without TLS) and the following properties and metadata:

```
{
  "serviceDefinitionName": "sysMonitor",
  "version": "1.0.0",
  "metadata": {
    "contract": "SysMonitor-AMQP-IDD-5.2.2",
    "schemaVersion": "1.0",
    "publisherMode": "native",
    "observationTypes": [
      "echo",
      "power",
      "status"
    ],
    "delivery": "persistent, publisher-confirmed"
  },
  "interfaces": [
    {
      "templateName": "generic_amqps",
      "protocol": "amqps",
      "policy": "CERT_AUTH",
      "properties": {
        "accessAddresses": [
          "amqp-broker"
        ],
        "accessPort": 5671,
        "exchange": "arrowhead",
        "baseRoutingKey": "TestCloud.sysmonitor.ConsumerAuthorization.ConsumerAuthorization",
        "operations": [
          "echo",
          "power",
          "status"
        ]
      }
    }
  ]
}
```

Listing 1: Registered service instance (TestCloud).

2 Service Operations

The publisher connects to the broker (`sysmonitor.broker.address`, default `amqp-broker`; port `sysmonitor.broker.port`, default 5671) with TLS, verifies the broker host name and authenticates with its X.509 system certificate (SASL `EXTERNAL`; the certificate CN is the broker user name). Every publication interval it publishes each observation to the topic exchange `arrowhead` with the routing key `<baseRoutingKey>.<operation>`, where the base routing key is `<Cloud>.sysmonitor.<System>.<serviceInstance>`. The local cloud name is taken from `sysmonitor.local.cloud` or the system certificate. Messages are persistent (delivery mode 2), `application/json`, carry the observation identifier as message id, and are publisher-confirmed (timeout 5 s); a failed cycle is retried with a new connection at the next interval. The broker's topic permissions allow the system to publish only below its own base routing key.

2.1 PUBLISH `<Cloud>.sysmonitor.ConsumerAuthorization.ConsumerAuthorization.echo`

Operation: echo
Output: Observation

Liveness: `status` (`alive` or `degraded`), `uptimeSeconds` and a `detail` text; quality `good` or `uncertain`.

2.2 PUBLISH `<Cloud>.sysmonitor.ConsumerAuthorization.ConsumerAuthorization.power`

Operation: power
Output: Observation

Power supply: `supplyType=line`, `state=available`; unit `1`.

2.3 PUBLISH `<Cloud>.sysmonitor.ConsumerAuthorization.ConsumerAuthorization.status`

Operation: status
Output: Observation

Process state: `state=running`, `health` (`healthy`/`degraded`) and `serviceCount`, the number of registered services.

```
{
  "schemaVersion": "1.0",
  "observationId": "ConsumerAuthorization-echo-1792670400000-42",
  "providerSystem": "ConsumerAuthorization",
  "serviceInstance": "ConsumerAuthorization",
  "observationType": "echo",
  "sourceTime": "2026-10-22T12:00:00.000Z",
  "sequence": 42,
  "value": {
    "status": "alive",
    "uptimeSeconds": 3600,
    "detail": "ConsumerAuthorization is running"
  },
  "quality": "good",
  "metadata": {
    "publisherMode": "native"
  }
```

```
18 }
```

Listing 2: An echo observation.

3 Data Models

3.1 struct Observation

Object Field	Value Type	Description
"schemaVersion"	String	`1.0`.
"observationId"	String	`<System>-<type>-<epoch ms>-<sequence>`.
"providerSystem"	String	The publishing system.
"serviceInstance"	String	Service instance, default the system name.
"observationType"	String	`echo`, `power` or `status`.
"sourceTime"	String	ISO 8601 UTC time with milliseconds.
"sequence"	Number	Sequence number per publisher.
"value"	Object	Observed value, see the operations.
"unit"	String	`1` for `power` and `status`; absent for `echo`.
"quality"	String	`good` or `uncertain`.
"metadata"	Object	`{"publisherMode": "native"}`.

3.2 Primitives

Type	Description
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] AMQP Working Group, “Amqp 0-9-1 specification,” AMQP Working Group, Tech. Rep., 2008.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “sysMonitor – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-sysMonitor-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

1 Interface overview

Mandatory mutual TLS, Arrowhead token/certificate binding, JSON request and response documents. Echo responses, orchestration subscription IDs, authorization public keys and encryption IV responses are text. Certificate-only operations and management are distinguished in the table.

1.1 HTTPS interface operations

HTTPS uses JSON unless an operation specifies text. Request bodies are limited to 1 MiB; upstream responses to 8 MiB. Identity-token format: `Bearer IDENTITY-TOKEN//token`. Certificate-only identity operations still require verified mutual TLS and apply credential/certificate binding.

Method	Path	Access
GET	/consumerauthorization/general/mgmt/get-config	management
POST	/consumerauthorization/general/mgmt/logs	management
GET	/consumerauthorization/monitor/echo	certificate
POST	/consumerauthorization/authorization/grant	identity
DELETE	/consumerauthorization/authorization/revoke/{instanceId}	identity
POST	/consumerauthorization/authorization/lookup	identity
POST	/consumerauthorization/authorization/verify	identity
POST	/consumerauthorization/authorization/mgmt/grant	management
DELETE	/consumerauthorization/authorization/mgmt/revoke	management
POST	/consumerauthorization/authorization/mgmt/query	management
POST	/consumerauthorization/authorization/mgmt/check	management
POST	/consumerauthorization/authorization-token/generate	identity
GET	/consumerauthorization/authorization-token/token/verify/{token}	identity
GET	/consumerauthorization/authorization-token/public-key	identity
POST	/consumerauthorization/authorization-token/encryption-key	identity
DELETE	/consumerauthorization/authorization-token/encryption-key	identity
POST	/consumerauthorization/authorization/mgmt/token/generate	management
POST	/consumerauthorization/authorization/mgmt/token/query	management
DELETE	/consumerauthorization/authorization/mgmt/token/revoke	management
POST	/consumerauthorization/authorization/mgmt/token/encryption-key	management
DELETE	/consumerauthorization/authorization/mgmt/token/encryption-key	management

Errors contain errorMessage, errorCode, exceptionType, origin and timestamp. Exact validation/error parity and

all Java security-policy settings remain release gates. Unknown routes return 404; unavailable authorities fail closed.

2 Limitations

Management-over-provider policy precedence, scoped policies, metadata checks, usage/time-limited tokens, Base64 and RSA JWT tokens, AES wire encryption and authenticated key wrapping are implemented. Foreign-consumer/federation token generation, Java database import, complete DTO/error parity and broad cryptographic interoperability fixtures remain incomplete. AES ECB/CBC are legacy Java wire formats, not recommendations for new protocols.